

WEEK-11

Views in SQL

- A view is a database object that has no values. Its contents are based on the base table.
- It contains rows and columns similar to the real table.
- In MySQL, the View is a **virtual table** created by a query by joining one or more tables.
- It is operated similarly to the base table but does not contain any data of its own.
- The View and table have one main difference that the views are definitions built on top of other tables (or views). If any changes occur in the underlying table, the same changes reflected in the View also.

The reasons for using Views”

- Views restrict access to the data because the view can display selective columns and rows from the table.
- Views provide groups of users with access to data according to their particular permissions.
- Views can be used to retrieve data from several tables, providing data independence for users.

Consider two tables: Student_table and Student_marks

Student_table

Stud_id	Name	Address
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_marks

Stud_id	Name	Marks	Age
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1. Creating view

- A view can be created using the **CREATE VIEW** statement.
- We can create a view from a **single table or multiple tables**.

Syntax:

```
CREATE [OR REPLACE] VIEW view_name AS  
SELECT column1, column2.....  
FROM table_name  
WHERE condition;
```

2. Creating View from a single table

- In this example, we create a View named DetailsView from the table Student_Detail.

Query:

```
CREATE VIEW DetailsView AS  
SELECT NAME, ADDRESS  
FROM Student_Details  
WHERE STU_ID < 4;
```

Just like table query, we can query the view to view the data.

```
SELECT * FROM DetailsView;
```

Output:

Name	Address
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

Note: Once the execution of the **CREATE VIEW** statement becomes successful, MySQL will create a view and stores it in the database.

3. Creating View from multiple tables

- View from multiple tables can be created by simply include multiple tables in the SELECT statement.
- In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

Query:

```
CREATE VIEW MarksView AS
```

```
SELECT Student_Detail.NAME, Student_Detail.ADDRESS, Student_Marks.MARKS
```

```
FROM Student_Detail, Student_Mark
```

```
WHERE Student_Detail.NAME = Student_Marks.NAME;
```

To display data of View MarksView:

```
SELECT * FROM MarksView;
```

Name	Address	Marks
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurugram	90

4. Deleting View

- A view can be deleted using the Drop View statement.

Syntax

```
DROP VIEW view_name;
```

Example:

If we want to delete the View **MarksView**, we can do this as:

```
DROP VIEW MarksView;
```

DCL- Data control language (DCL)

- DCL- Data control language (DCL) is used to manage database security and access control.
- It is mainly used for revoke and to grant the user the required access to a database.
- It is a part of the structured query language (SQL).
 - It helps in controlling access to information stored in a database.
 - It is the simplest among three commands.
 - It provides the administrators, to remove and set database permissions to desired users as needed.
 - These commands are employed to grant, remove and deny permissions to users for retrieving and manipulating a database.

Difference between system and object privileges

- In a database, every user account can be granted a number of *privileges*, which are also known as *permissions*.
- These privileges allow a particular user account to do certain things, like DELETE and UPDATE certain tables, CREATE a database, SELECT from a certain table, and many other things.
- Privileges are further divided into two different categories:
 - 1. system privileges and
 - 2. object privileges.

System privileges

System privileges are privileges given to users to allow them to perform certain functions that deal with *managing the database and the server* . Most of the different types of permissions supported by the database vendors fall under the system privilege category

Examples of Oracle system privileges

- **CREATE USER.** The **CREATE USER** permission, when granted to a database user, allows that database user to create new users in the database.
- **CREATE TABLE.** The **CREATE TABLE** permission, when granted to a database user, allows that database user to create tables in their own schema. This type of privilege is also available for other object types – like stored procedures and indexes.
- **CREATE SESSION.** The **CREATE SESSION** permission, when granted to a database user, allows that database user to connect to the database.

Object privileges

- Object privileges are privileges given to users so that they can perform certain actions upon certain database objects – where database objects are things like tables, stored procedures, indexes, etc.
- Some examples of object privileges include granting a particular database user the right to **DELETE** and/or **SELECT** from a particular table. This is done using the **GRANT** clause.

Hence, system privileges are used for server and database privileges. But object privileges are used to grant privileges on database objects like tables, stored procedures, indexes, etc.

USER CREATION IN MYSQL :

- The MySQL user is a record in the **USER** table of the MySQL server that contains the login information, account privileges, and the host information for MySQL account. It is essential to create a user in MySQL for accessing and managing the databases.
- The MySQL Create User statement allows us to create a new user account in the database server.
- Syntax : **CREATE USER [IF NOT EXISTS] account_name IDENTIFIED BY 'password';**
- Example: **CREATE USER peter@localhost IDENTIFIED BY 'jtp12345';**

Or

CREATE USER IF NOT EXISTS adam@localhost IDENTIFIED **BY** 'jtp123456';

DCL Commands

The Data Control Language (DCL) commands are as follows –

GRANT Command

It is used to grant a privilege(permissions) to a user. GRANT command allows specified users to perform specified tasks.

Syntax

```
GRANT privilege_name(s)  
ON object  
TO user_account_name;
```

Here,

- **privilege_name(s):** It specifies the access rights or grant privilege to user accounts. If we want to give multiple privileges, then use a comma operator to separate them.
- **object** :It determines the privilege level on which the access rights are being granted. It means granting privilege to the table; then the object should be the name of the table.
- **user_account_name** : It determines the account name of the user to whom the access rights would be granted.

Privilege Levels

MySQL supports the following privilege levels:

Privilege Level	Syntax	Descriptions
Global	GRANT ALL ON *.* TO john@localhost;	It applies to all databases on MySQL server. We need to use *.* syntax for applying global privileges. Here, the user can query data from all databases and tables of the current server.

Database	GRANT ALL ON mydb.* TO john@localhost;	It applies to all objects in the current database. We need to use the db_name.* syntax for applying this privilege. Here, a user can query data from all tables in the given database.
Table	GRANT DELETE ON mydb.employees TO john@localhsot;	It applies on all columns in a specified table. We need to use db_name.table_name syntax for assigning this privilege. Here, a user can query data from the given table of the specified database.

REVOKE Command

The revoke statement enables system administrators to revoke privileges and roles to the MySQL user accounts so that they cannot use the assigned permission on the database in the past.

Syntax

```
REVOKE privilege_name(s)
ON object
FROM user_account_name;
```

Here,

- **privilege_name(s):** It specifies the access rights or grant privilege that we want to revoke from user accounts.
- **object :** It determines the privilege level on which the access rights are being granted. It means granting privilege to the table; then the object should be the name of the table.
- **user_account_name :** It determines the account name of the user from which we want to revoke the access rights.Example

Example:

1) **REVOKE UPDATE, INSERT ON** mystudentdb.* **FROM** john@localhost;

2) **REVOKE ALL, GRANT OPTION FROM** john@localhost;